

Estimating Population Range by Recurring Online Chunk Bootstrap with Non-cumulative Data in Streaming Data Environment

Abstract—

*Index Terms—*Article submission, IEEE, IEEEtran, journal, L^AT_EX, paper, template, typesetting.

I. INTRODUCTION

II. RELATED WORKS AND CONCEPTS OF PREDICTING NUMBER OF CLUSTERS

This section summarized some interesting works relating to predicting the number of clusters based on distance analysis among data points.

III. STUDIED PROBLEM AND OBJECTIVES

Let a chunk of integer data, a set of bins, and a set of integer data be defined as follows:

- 1) A set of integer data, $\mathbf{D} = \{a \leq d_j \leq b \mid 1 \leq j \leq p\}$, for integer constants p , a , and b .
- 2) An integer data chunk $\mathbf{C}$ divided into n smaller chunks, $\mathbf{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_i, \dots, \mathbf{c}_n \mid 1 \leq n < \infty\}$. Each chunk has integers randomly taken from $\mathbf{D}$.
- 3) A bin $\mathbf{B}$ for containing incoming all integers in each chunk. There are two attributes, $v_i^{(min)}$ and $v_i^{(max)}$, defining the interval of integer values such that any integer $v_i^{(min)} \leq d_j \leq v_i^{(max)}$ can be assigned to this bin.

Each chunk $\mathbf{c}_i$ sequentially flows into the bin. If some integers whose values are less than $v_1^{(min)}$ or larger than $v_m^{(max)}$, then the width of $\mathbf{B}$ must be expanded.

In the beginning, the size of bin $\mathbf{B}$ is made large enough to contain all integers in the first incoming chunk $\mathbf{c}_1$. Then size of $\mathbf{B}$ is occasionally expanded so that all integers in the other next incoming chunks can be assigned to the intervals of $\mathbf{B}$. Bin $\mathbf{B}$ is expanded if the values of some integers in some incoming chunks are either less than $v^{(min)}$ or larger than $v^{(max)}$. Therefore, the studied problem is defined as follows.

Let $\min(\mathbf{D})$ and $\max(\mathbf{D})$ be the minimum and maximum values of $\mathbf{D}$. After capturing of integers in the first chunk, how to achieve the minimum number of expansions of $\mathbf{B}$ so that

- 1) All incoming integers in the next other chunks can be assigned to $\mathbf{B}$.

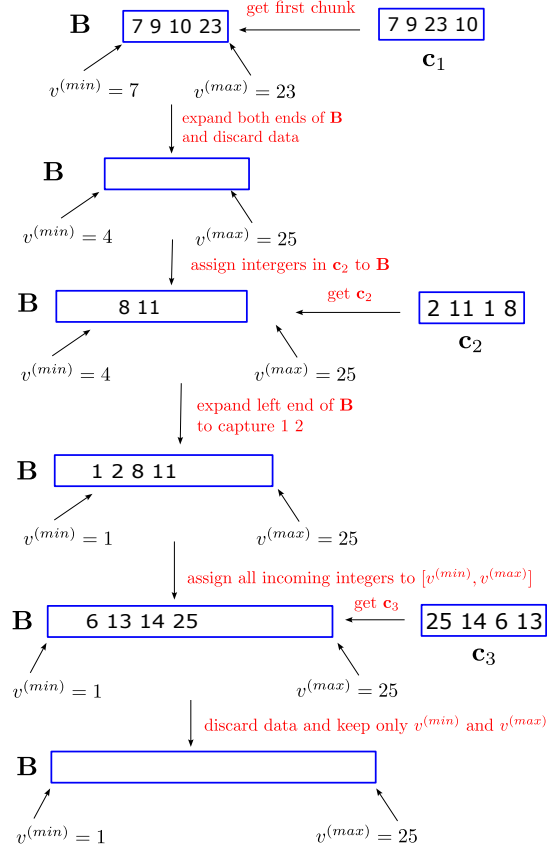


Fig. 1. An example of studied problem.

- 2) $(\min(\mathbf{D}) - v_1^{(min)}) \geq 0$ is minimum.
- 3) $(v_i^{(max)} - \max(\mathbf{D})) \geq 0$ is minimum.

Figure 1 illustrates an example scenario of capturing chunks. There are 3 sequentially incoming chunks containing these integers: $\mathbf{c}_1 = \{7, 9, 23, 10\}$, $\mathbf{c}_2 = \{2, 11, 1, 8\}$, $\mathbf{c}_3 = \{25, 14, 6, 13\}$. After capturing $\mathbf{c}_1$, the values of left and right ends of bin $\mathbf{B}$ are set to $v^{(min)} = 7$ and $v^{(max)} = 23$ and all data are discarded. Then, both ends are expanded in advance to $v^{(min)} = 4$ and $v^{(max)} = 25$, preparing for $\mathbf{c}_2$. When $\mathbf{c}_2$ enters, all integers except 2 can be captured because the left end value $v^{(min)} = 4$ is larger than 2. Thus, the left end $v^{(min)}$ is expanded to $v^{(min)} = 2$. No need to expand the right end. All data in $\mathbf{c}_2$ are discarded. Then, get $\mathbf{c}_3$. The interval of $\mathbf{B}$ is large enough to capture all integers of $\mathbf{c}_3$. After capturing $\mathbf{c}_3$, all integers are discarded. In this example, the number of expansions is 2, after chunks 1 and 2. Generally, how to

achieve the minimum number of expansions of $\mathbf{B}$ in advance so that both expanded values are also minimum.

A. Constraints

The amount of integers in each $\mathbf{c}_i$ is denoted by $|\mathbf{c}_i|$. Bin B can be considered as an interval having left end denoted by L and right end denoted by R . The following constraints are imposed.

- 1) The probability of distribution of each integer d_j in $\mathbf{c}_i$ is unknown.
- 2) $|\mathbf{c}_i| \neq |\mathbf{c}_{i+1}|$ or $|\mathbf{c}_i| = |\mathbf{c}_{i+1}|$.
- 3) After assigning all integers of $\mathbf{c}_i$ inside the bin, chunk $\mathbf{c}_i$ is completely discarded and never reentered the bin assignment.

IV. CONCEPT OF RECURRING ONLINE BOOTSTRAP IN STREAMING ENVIRONMENT WITH UNRECORDED DATA

A. Algorithm

explain standard histogram first

The algorithm has two phases. The first phase

Algorithm 1: Capturing data chunks and expanding $\mathbf{B}$ when it is necessary.

Input: Stream of data chunks, $\mathbf{c}_i$ for $1 \leq i < \infty$.

Output:

Phase 1: Capturing $\mathbf{c}_1$ and get initial expansion of $\mathbf{B}$.

1. Get $\mathbf{c}_1$ and set $total_data = |\mathbf{c}_1|$.
2. Let $v^{(min)} = \min(\mathbf{c}_1)$ and $v^{(max)} = \max(\mathbf{c}_1)$.
3. Divide $\mathbf{B}$ into 8 equal sub-intervals $\mathbf{b}_i$ for $1 \leq i \leq 8$, each of size $(v^{(max)} - v^{(min)})/8$.
4. Put the integers in $\mathbf{c}_1$ whose values are within sub-intervals $\mathbf{b}_1$ and $\mathbf{b}_8$ into these two sub-intervals.
5. Count the number of integers in $\mathbf{b}_1$ and $\mathbf{b}_8$ and let $|\mathbf{b}_1|$ and $|\mathbf{b}_8|$ denote these numbers.
6. Let $avg = (v^{(max)} + v^{(min)})/2$ be the middle value of $\mathbf{B}$.
7. Find the types of probability distribution in list P best fitting the data in $\mathbf{b}_1$, and $\mathbf{b}_8$ by using Algorithm 2.1 with $total_data$, $\mathbf{b}_1$, and $\mathbf{b}_8$.
8. Compute the standard number of integers in $\mathbf{b}_1$ denoted by $lstd$, and in $\mathbf{b}_8$ denoted by $rstd$. from the best fitted probability distribution by using Algorithm 2.2.
9. **While** $|\mathbf{b}_1| > lstd$ or $|\mathbf{b}_8| > rstd$ **do**
10. **If** $|\mathbf{b}_1| > lstd$ **then**
11. Expand $v^{(min)}$ by using Algorithm 3 with all integers in $\mathbf{b}_1$.
12. **EndIf**
13. **If** $|\mathbf{b}_8| > rstd$ **then**
14. Expand $v^{(max)}$ by using Algorithm 4 with all integers in $\mathbf{b}_8$.
15. **EndIf**
16. Adjust the width of $\mathbf{b}_1$ and $\mathbf{b}_8$ by dividing $\mathbf{B}$

into 8 equal sub-intervals $\mathbf{b}_i$ for $1 \leq i \leq 8$, each of size $(v^{(max)} - v^{(min)})/8$.

17. **EndWhile**

18. Discard $\mathbf{c}_1$ and all integers in $\mathbf{b}_1$ and $\mathbf{b}_8$.

Phase 2: Capturing other $\mathbf{c}_i$ and determining the necessity of expanding $\mathbf{B}$.

1. **while** there exists a new incoming chunk $\mathbf{c}_i$ **do**
2. $total_data = total_data + |\mathbf{c}_i|$.
3. **If** $\min(\mathbf{c}_i) < v^{(min)}$ **then**
4. $v^{(min)} = \min(\mathbf{c}_i)$.
5. **EndIf**.
6. **If** $v^{(max)} < \max(\mathbf{c}_i)$ **then**
7. $v^{(max)} = \max(\mathbf{c}_i)$.
8. **EndIf**.
9. Divide $\mathbf{B}$ into 8 equal sub-intervals $\mathbf{b}_i$ for $1 \leq i \leq 8$, each of size $(v^{(max)} - v^{(min)})/8$.
10. Put the integers in $\mathbf{c}_i$ whose values are within sub-intervals $\mathbf{b}_1$ and $\mathbf{b}_8$ into these two sub-intervals.
11. Count the number of integers in $\mathbf{b}_1$ and $\mathbf{b}_8$ and let $|\mathbf{b}_1|$ and $|\mathbf{b}_8|$ denote these numbers.
12. Find the types of probability distribution in list P best fitting the data in $\mathbf{b}_1$, and $\mathbf{b}_8$ by using Algorithm 2.1 with $total_data$, $\mathbf{b}_1$, and $\mathbf{b}_8$.
13. Compute the standard number of integers in $\mathbf{b}_1$ denoted by $lstd$, and in $\mathbf{b}_8$ denoted by $rstd$ from the best fitted probability distribution by using Algorithm 2.2.
14. **While** $|\mathbf{b}_1| > lstd$ or $|\mathbf{b}_8| > rstd$ **do**
15. **If** $|\mathbf{b}_1| > lstd$ **then**
16. Expand $v^{(min)}$ by using Algorithm 3 with all integers in $\mathbf{b}_1$.
17. **EndIf**
18. **If** $|\mathbf{b}_8| > rstd$ **then**
19. Expand $v^{(max)}$ by using Algorithm 4 with all integers in $\mathbf{b}_8$.
20. **EndIf**
21. Adjust the width of $\mathbf{b}_1$ and $\mathbf{b}_8$ by dividing $\mathbf{B}$ into 8 equal sub-intervals $\mathbf{b}_i$ for $1 \leq i \leq 8$, each of size $(v^{(max)} - v^{(min)})/8$.
22. **EndWhile**.
23. Discard $\mathbf{c}_i$ and all integers in $\mathbf{b}_1$ and $\mathbf{b}_8$.
24. **EndWhile**.

Algorithm 2.1: Finding the types of probability distribution in list P best fitting the data in $\mathbf{b}_1$, and $\mathbf{b}_8$.

Input: (1) A list of standard probability distribution P . (2) $total_data$. (3) $\mathbf{b}_1$. (4) $\mathbf{b}_8$.

Output: $lname$ and $rname$.

1. **For** each type probability distribution $p \in P$ **do**
2. Divide the area under standard probability distribution p into 8 stripes of equal width.
3. Let $l_4^{(p)}$ be the percentage of data in the 4th stripe

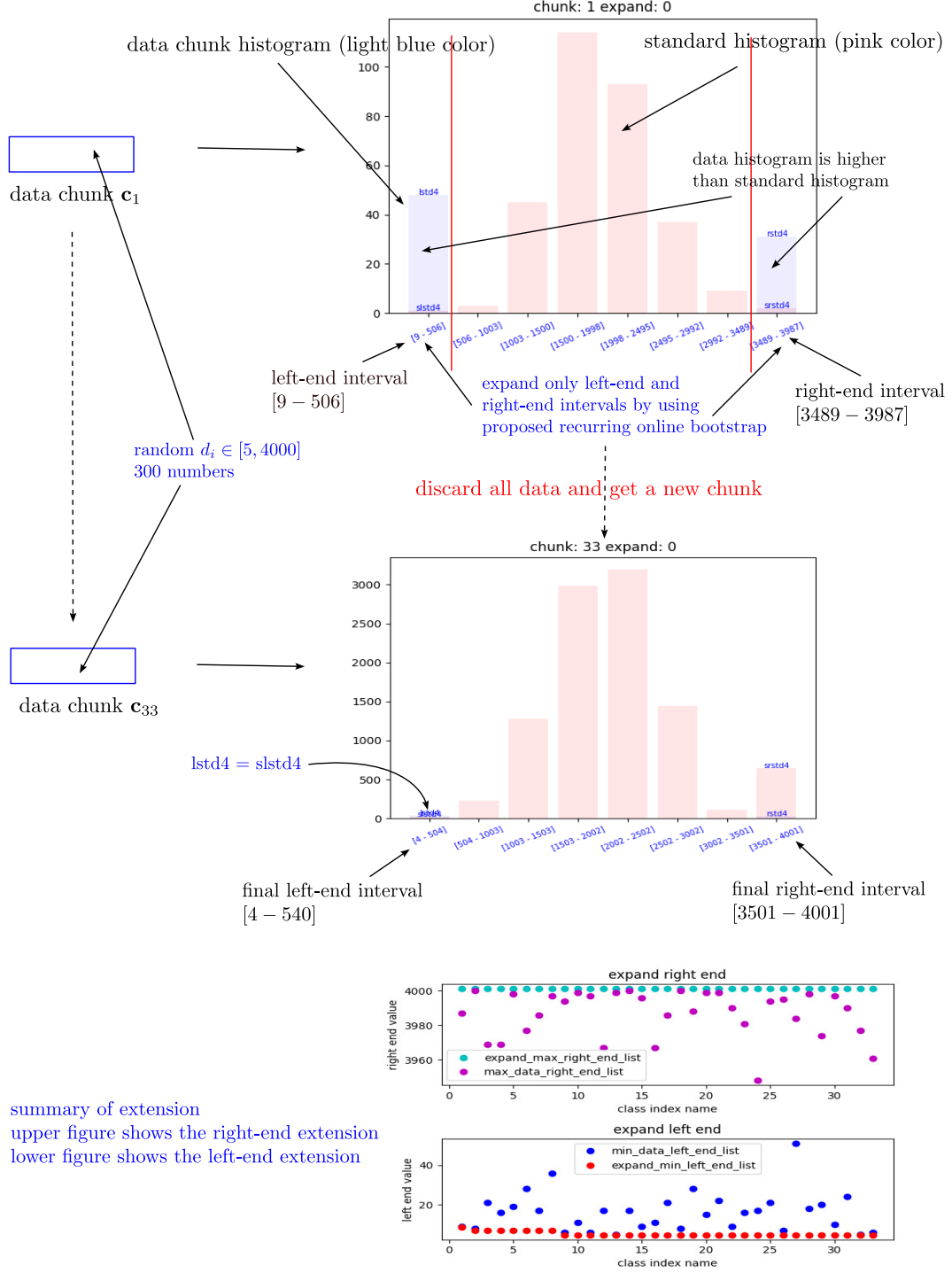


Fig. 2. Framework.

4. Let $r_4^{(p)}$ be the percentage of data in the 4^{th} stripe to the right of mean.
5. Compute the difference between standard number of integers in the 4^{th} left stripe and $|b_1|$:
 $ld^{(p)} = abs(l_4^{(p)} * total_data - |b_1|)$.
6. Compute the difference between standard number

of integers in the 4^{th} right stripe and $|b_8|$:
 $rd^{(p)} = abs(r_4^{(p)} * total_data - |b_8|)$.

7. **EndFor**
8. Find $lname = \arg \min_{p \in P} (ld^{(p)})$.
9. Find $rname = \arg \max_{p \in P} (rd^{(p)})$.
10. **Return** $lname$ and $rname$.

Algorithm 2.2: Computing $lstd$ and $rstd$.

Input: (1) A list of standard probability distribution P . (2) $total_data$. (3) $lname$. (4) $rname$. (5) $l_4^{(p)}$ and $r_4^{(p)}$; $\forall p \in P$ from Algorithm 2.1.

Output: $lstd$ and $rstd$.

1. **If** $lname$ is the same as $rname$ **then**
2. Set $lstd = l_4^{(lname)} * total_data$.
3. Set $rstd = r_4^{(rname)} * total_data$.
4. **EndIf**
5. **If** $lname$ is different from $rname$ **then**
6. Set $lstd = \max_{p \in P} (l_4^{(p)}) * total_data$.
7. Set $rstd = \max_{p \in P} (r_4^{(p)}) * total_data$.
8. **EndIf**
9. **Return** $lstd$ and $rstd$.

Algorithm 3: Recurring online chunk bootstrap for B_1 .

Input: (1) Present set of incoming integers in B_1 ; (2) Number of bootstrap iterations N ; (3) $mean(a)$ is a function computing the mean of set a ; (4) $std(a)$ is a function computing the standard deviation of set a ; (5) $abs(x)$ is the absolute value of constant x .

Output: $v^{(min)}$.

1. Let $S = \emptyset$ be a set of bootstrapped samples.
2. Let $M = \emptyset$ be a set of mean of each bootstrapped sample.
3. Let $P = \emptyset$ be a set of standard deviation of each bootstrapped sample.
4. $prev_mean = 0$.
5. **For** $1 \leq i \leq N$ **do**:
6. Let s_i be a set of randomly sampled integers of size $|B_1|$ from B_1 with replacement.
7. $S = S \cup \{s_i\}$.
8. $present_mean = (mean(s_i) + prev_mean)/2$.
9. $prev_mean = present_mean$.
10. $M = M \cup \{present_mean\}$.
11. **EndFor**.
12. $\mu^{(boot)} = mean(M)$.
13. **For** each $s_i \in S$ **do**
14. $P = P \cup \{std(s_i)\}$.
15. **EndFor**
16. $\sigma^{(boot)} = mean(P)$.
17. $\mu^{(diff)} = abs(mean(B_1) - \mu^{(boot)})$.
18. $\sigma^{(diff)} = abs(std(B_1) - \sigma^{(boot)})$.
19. **If** $mu^{(boot)} < mean(B_1)$ **do**
20. $v^{(min)} = min(B_1) - \mu^{(diff)}$.
21. **If** $mean(B_1) < mu^{(boot)}$ **do**
22. $v^{(min)} = min(B_1) - \sigma^{(diff)}$.

Algorithm 4: Recurring online chunk bootstrap for B_8 .

Input: (1) Present set of incoming integers in B_1 ; (2) Number of bootstrap iterations N ; (3) $mean(a)$ is a function computing the mean of set a ; (4) $std(a)$ is a function computing the standard deviation of set a ; (5) $abs(x)$ is the absolute value of constant x .

Output: $v^{(max)}$.

1. Let $S = \emptyset$ be a set of bootstrapped samples.
2. Let $M = \emptyset$ be a set of mean of each bootstrapped sample.
3. Let $P = \emptyset$ be a set of standard deviation of each bootstrapped sample.
4. $prev_mean = 0$.
5. **For** $1 \leq i \leq N$ **do**:
6. Let s_i be a set of randomly sampled integers of size $|B_8|$ from B_8 with replacement.
7. $S = S \cup \{s_i\}$.
8. $present_mean = (mean(s_i) + prev_mean)/2$.
9. $prev_mean = present_mean$.
10. $M = M \cup \{present_mean\}$.
11. **EndFor**.
12. $\mu^{(boot)} = mean(M)$.
13. **For** each $s_i \in S$ **do**
14. $P = P \cup \{std(s_i)\}$.
15. **EndFor**
16. $\sigma^{(boot)} = mean(P)$.
17. $\mu^{(diff)} = abs(mean(B_8) - \mu^{(boot)})$.
18. $\sigma^{(diff)} = abs(std(B_8) - \sigma^{(boot)})$.
19. **If** $mu^{(boot)} < mean(B_8)$ **do**
20. $v^{(max)} = min(B_8) + \sigma^{(diff)}$.
21. **If** $mean(B_1) < mu^{(boot)}$ **do**
22. $v^{(max)} = min(B_8) + \mu^{(diff)}$.

REFERENCES

- [1] Hong Yu, Zhanguo Liu, Guoyin Wang, "An automatic method to determine the number of clusters using decision-theoretic rough set", International Journal of Approximate Reasoning 55 (2014) 101–115.